Trabajo Práctico 3

1. Responder breve y claramente:

a. Probar que $n^3 = O(2^n)$.

1. Por definición, $n^3 = O(2^n)$ si y solo si para todo $n \geq n_0$, existe una constante $c > 0$ tal que $n^3 \leq c \cdot 2^n$.
2. Planteamos la desigualdad: $n^3 \leq c \cdot 2^n$.
3. Asumimos $c = 1$ para simplificar, entonces necesitamos demostrar que $n^3 \leq 2^n$ para todo $n \geq n_0$.
4. Si $n = 10$, entonces $n^3 = 1000$ y $2^n = 1024$, por lo que $n^3 \leq 2^n$.
5. Si $n = 11$, entonces $n^3 = 1331$ y $2^n = 2048$, por lo que $n^3 \leq 2^n$.
6. ...
7. Para $n \geq 10$, la función 2^n crece mucho más rápido que n^3, por lo que la desigualdad se mantiene.
8. Por lo tanto, podemos concluir que $n^3 = O(2^n)$ con $n_0 = 10$ y $c = 1$.

b. Probar que si $T_1(n) = O(T_2(n))$, entonces $TIME(T_1(n)) \subseteq TIME(T_2(n))$.

1. Sea $L \in TIME(T_1(n))$. Por definición, existe una MT M que decide L en tiempo $O(T_1(n))$.
2. Como se sabe por hipótesis que $T_1(n) = O(T_2(n))$, se tiene que $O(T_1(n)) \subseteq O(T_2(n))$.
3. Por lo tanto M también decide L en tiempo $O(T_2(n))$, lo que implica que $L \in TIME(T_2(n))$.

Intuitivamente, si un algoritmo puede resolver un problema en tiempo $T_1(n)$, y $T_1(n)$ es asintóticamente menor o igual a $T_2(n)$, entonces ese mismo algoritmo también puede resolver el problema en tiempo $T_2(n)$, ya que $T_2(n)$ es una cota superior para el tiempo de ejecución del algoritmo.

c. ¿Qué formulan la Tesis de Church-Turing y la Tesis Fuerte de Church-Turing?

1. La Tesis de Church-Turing formula que cualquier función computable por un algoritmo puede ser computada por una Máquina de Turing.
2. La Tesis Fuerte de Church-Turing formula que si un lenguaje L es decidible en tiempo polinomial por un modelo computacional físicamente realizable, entonces también es decidible en tiempo polinomial por una MT.

d. ¿Por qué si un lenguaje pertenece a P también su complemento pertenece a P?

1. Sea $L \in P$. Por definición, existe una MT M que decide L en tiempo polinomial.
2. Se puede construir una MT M' que simula a M y acepta la cadena de entrada w si M la rechaza, y rechaza w si M la acepta. Por lo tanto M' decide a L^C en tiempo polinomial.
3. Por lo tanto, $L^C \in P$.

e. Sea L un lenguaje de NP. Explicar por qué los certificados de L miden un tamaño polinomial con respecto al tamaño de las cadenas de entrada.

1. Por definición, un lenguaje $L \in NP$ si existe una MT que lo **verifica** en tiempo polinomial con la ayuda de un certificado.
2. Esto implica que para cada cadena de entrada w, el certificado x que se utiliza para verificar si $w \in L$ debe tener un tamaño que es a lo sumo polinomial en relación al tamaño de w.
3. Si el certificado tuviera un tamaño mayor que polinomial, entonces la verificación no podría hacerse en tiempo polinomial y tendríamos una contradicción, porque se viola la definición de NP.

Por lo tanto, los certificados de un lenguaje en NP deben medir un tamaño polinomial con respecto al tamaño de las cadenas de entrada.

f. Probar que $NP \neq \text{CO-NP}$ implica $P \neq NP$.

1. Por contrarrecíproco $(p \to q \equiv \neg q \to \neg p)$, reescribimos la expresión: $P = NP$ implica $NP = \text{CO-NP}$.

2. Si $P = NP$, entonces todos los lenguajes en NP también están en P.
3. Se sabe que P es cerrado bajo complemento, lo que significa que si un lenguaje está en P, su complemento también está en P.
4. Por lo tanto, si $P = NP$, entonces NP también es cerrado bajo complemento, lo que implica que $NP = \text{CO-NP}$.

g. Mostrar un esquema de prueba de la transitividad de las reducciones polinomiales.

1. Definición de reducción polinomial: $L_1 \leq_p L_2$ si existe una función total computable en tiempo polinomial f tal que para toda cadena w, $w \in L_1$ si y solo si $f(w) \in L_2$.
2. Definición de transitividad: Si $L_1 \leq_p L_2$ y $L_2 \leq_p L_3$, entonces $L_1 \leq_p L_3$.
3. Como se sabe que $L_1 \leq_p L_2$ (por hipótesis), existe una función f que reduce L_1 a L_2.
4. Como se sabe que $L_2 \leq_p L_3$ (por hipótesis), existe una función g que reduce L_2 a L_3.
5. Se puede construir una función h que reduzca L_1 a L_3 definiendo $h(w) = g(f(w))$, es decir, usando composición de funciones.
6. Como la composición de funciones computables en tiempo polinomial también es computable en tiempo polinomial, h es una función total computable en tiempo polinomial.
7. Por lo tanto, $L_1 \leq_p L_3$, lo que demuestra la transitividad de las reducciones polinomiales.

h. Probar que si $L_1 \leq_p L_2$ y $L_2 \in P(NP)$, entonces $L_1 \in P(NP)$. En otras palabras, L_2 es tan o más difícil que L_1, en el marco de la complejidad temporal.

1. Hipótesis: $L_1 \leq_p L_2$ y $L_2 \in P$.
2. Por definición de reducción polinomial, existe una función total computable en tiempo polinomial f tal que para toda cadena w, $w \in L_1$ si y solo si $f(w) \in L_2$.
3. Dado que $L_2 \in P$, existe una MT M que decide L_2 en tiempo polinomial.
4. Se puede construir una MT M' que dada una cadena de entrada w primero calcula $f(w)$, lo cual se hace en tiempo polinomial, y luego simula a M con la entrada $f(w)$. El tiempo total de M' es la suma del tiempo para calcular $f(w)$ y el tiempo para simular a M, ambos de los cuales son polinomiales. Como la suma de dos funciones polinomiales es también una función polinomial, M' decide L_1 en tiempo polinomial.
5. Por lo tanto, $L_1 \in P$.

i. ¿Por qué si $P \neq NP$, un lenguaje NP-completo no pertenece a P?

1. Por definición, un lenguaje L es NP-completo si $L \in NP$ y además para todo lenguaje $L' \in NP$, $L' \leq_p L$.
2. Si $P \neq NP$, entonces existen lenguajes que están en NP pero no están en P.
3. Si un lenguaje NP-completo L estuviera en P, entonces por la definición de NP-completo, todos los lenguajes en NP se reducirían polinomialmente a L, lo que implicaría que todos los lenguajes en NP estarían en P.
4. Esto contradice la hipótesis de que $P \neq NP$.

j. ¿Cuándo se sospecha que un lenguaje de NP está en NPI?

La clase NPI (NP-Intermedio) se refiere a los lenguajes que están en NP pero no son NP-completos y además no están en P. Se sospecha que un lenguaje de NP está en NPI cuando no se ha hallado una reducción polinomial desde ningún lenguaje NP-completo hacia él, y tampoco se ha encontrado un algoritmo de tiempo polinomial para decidirlo.

2. Sea $SMALL - SAT = \{\phi \mid \phi \text{ es una fórmula booleana tal que...}\}$. No tiene cuantificadores, está en la forma normal conjuntiva (o FNC), y existe una asignación de valores de verdad que la satisface en la que hay a lo sumo 3 variables con valor de verdad V. Probar que $\text{SMALL-SAT} \in P$. Comentario: una fórmula booleana sin cuantificadores está en la forma FNC si es una conjunción de disyunciones de variables o variables negadas, como es el caso, por ejemplo, de la fórmula $(x_1 \vee x_2) \wedge x_4 \wedge$

$(\neg x_3 \vee x_5 \vee x_6)$. Ayuda: Una MT que decida SMALL-SAT debe contemplar asignaciones con cero, uno, dos y hasta tres valores de verdad verdadero.

Para probar que un lenguaje está en P, se debe construir una MT M que decida el lenguaje en tiempo polinomial. M puede funcionar de la siguiente manera:

1. Recibe una fórmula booleana ϕ como cadena de entrada, donde k es la cantidad de variables distintas que aparecen en ϕ.
2. Chequea que la fórmula ϕ sea válida, es decir, que esté en la forma normal conjuntiva (FNC) y que no tenga cuantificadores. Esto se puede hacer en tiempo polinomial con respecto al tamaño de ϕ porque es un chequeo sintáctico sencillo que se realiza símbolo a símbolo y cláusula a cláusula, teniendo en cuenta la longitud de la fórmula $|\phi| = n$.
3. Si la fórmula es válida, se tienen que chequear los 4 casos:
   i. Le asigna el valor de verdad falso a **todas** las variables de ϕ y verifica si ϕ se satisface. En el peor caso tenemos que recorrer toda la fórmula para verificar que se satisface, lo cual se hace en tiempo $O(n)$. Si se satisface, entonces M acepta. Si no se satisface, entonces M continúa con el siguiente caso.
   ii. Ahora tenemos que asignar el valor de verdad verdadero a una sola variable de ϕ y falso a las demás, y verificar si ϕ se satisface. Pero tenemos que hacerlo con cada una de las variables de ϕ que es k, para lo cual podemos usar la fórmula matemática de combinatoria $\binom{k}{1} = k$. Como cada chequeo toma tiempo $O(n)$, el tiempo total para este caso es $O(k \cdot n)$.
   iii. Ahora tenemos que asignar el valor de verdad verdadero a dos variables de ϕ y falso a las demás, y verificar si ϕ se satisface. Por lo tanto tenemos $\binom{k}{2} = \frac{k!}{2!(k-2)!} = \frac{k(k-1)}{2} = O(k^2)$. Como cada chequeo toma tiempo $O(n)$, el tiempo total para este caso es $O(k^2 \cdot n)$.
   iv. Finalmente tenemos que asignar verdadero a tres variables de ϕ y falso a las demás, y verificar si ϕ se satisface. Por lo tanto tenemos $\binom{k}{3} = \frac{k!}{3!(k-3)!} = \frac{k(k-1)(k-2)}{6} = O(k^3)$. El tiempo total es $O(k^3 \cdot n)$.
4. Por lo tanto la complejidad total es $O(n) + O(k \cdot n) + O(k^2 \cdot n) + O(k^3 \cdot n) = O(k^3 \cdot n)$, que es polinomial con respecto al tamaño de la fórmula ϕ.

En conclusión, M decide SMALL-SAT en tiempo polinomial, por lo que $\text{SMALL-SAT} \in P$.

3. Dados los dos lenguajes siguientes, (1) justificar por qué no estarían en P, (2) probar que están en NP, (3) justificar por qué sus complementos no estarían en NP:

a. El problema del conjunto dominante de un grafo consiste en determinar si un grafo no dirigido tiene un conjunto dominante de vértices. Un subconjunto D de vértices de un grafo G es un conjunto dominante de G, sii todo vértice de G fuera de D es adyacente a algún vértice de D. El lenguaje que representa el problema es $\text{DOM-SET} = \{(G, K) \mid$ $\text{G es un grafo ND y tiene un conjunto dominante de K vértices}\}$.

1. El problema dado no estaría en P porque para decidir si un grafo G tiene un conjunto dominante de K vértices, se tendrían que verificar todas las combinaciones posibles de K vértices en el grafo, lo cual es una tarea que crece exponencialmente con el número de vértices del grafo. Por lo tanto, no existe un algoritmo de tiempo polinomial (que se sepa) para resolver este problema.
2. El problema está en NP porque se puede verificar si un subconjunto D de vértices (que es el certificado) es un conjunto dominante de G en tiempo polinomial. Para verificar esto, se puede recorrer cada vértice fuera de D y comprobar si es adyacente a algún vértice en D. Si todos los vértices fuera de D son adyacentes a algún vértice en D, entonces el certificado es válido y se acepta la cadena de entrada.
3. El complemento del problema sería determinar si un grafo NO tiene un conjunto dominante de K vértices. Para verificar esto, se tendría que comprobar que para cada subconjunto de K vértices, existe al menos un vértice fuera del subconjunto que no es adyacente a ningún vértice dentro del subconjunto. Para verificar esto, parecería necesario recibir un certificado que capture esta información, lo cual sugiere que dicho certificado sería de tamaño exponencial. Como en NP los certificados deben ser de tamaño **polinomial** respecto de la entrada, no se conoce una forma de verificar este problema en tiempo polinomial mediante un certificado de tamaño polinomial. Por lo tanto, parecería que el complemento del problema no pertenece a NP.

b. El problema de los grafos isomorfos consiste en determinar si dos grafos son isomorfos. Dos grafos son isomorfos si son idénticos salvo por la denominación de sus arcos. Por ejemplo, el grafo $G_1 =$

$(\{1,2,3,4\},\{(1,2),(2,3),(3,4),(4,1)\})$ es isomorfo al grafo $G_2 = (\{1,2,3,4\},\{(1,2),(2,4),(4,3),(3,1)\})$. El lenguaje que representa el problema es $ISO = \{(G_1, G_2) \mid G_1 \text{ y } G_2 \text{ son grafos isomorfos}\}$.

1. El problema dado no estaría en P porque para decidir si dos grafos G_1 y G_2 son isomorfos, se tendrían que verificar todas las posibles correspondencias entre los vértices de G_1 y los vértices de G_2, lo cual es una tarea que crece de forma factorial con el número de vértices de los grafos. Por lo tanto, no existe un algoritmo de tiempo polinomial (que se sepa) para resolver este problema.
2. El problema está en NP porque se puede verificar si una correspondencia dada entre los vértices de G_1 y G_2 es un isomorfismo en tiempo polinomial. Para verificar esto, se puede recorrer cada arista de G_1 y comprobar si existe una arista correspondiente en G_2 según la correspondencia dada. Si todas las aristas de G_1 tienen una arista correspondiente en G_2, entonces el certificado es válido y se acepta la cadena de entrada.
3. El complemento del problema sería determinar si dos grafos NO son isomorfos. Para verificar esto, se tendría que comprobar que para cada posible correspondencia entre los vértices de G_1 y G_2, existe al menos una arista en G_1 que no tiene una arista correspondiente en G_2. Para verificar esto, parecería necesario recibir un certificado que capture esta información, lo cual sugiere que dicho certificado sería de tamaño exponencial. Al igual que en el punto anterior, como en NP los certificados deben ser de tamaño **polinomial** respecto de la entrada, no se conoce una forma de verificar este problema en tiempo polinomial mediante un certificado de tamaño polinomial. Por lo tanto, parecería que el complemento del problema no pertenece a NP.

4. Se prueba que $NP \subseteq EXP$. La prueba es la siguiente. Si $L \in NP$, entonces existe una MT M que, para toda cadena de entrada w, verifica en tiempo $poly(|w|)$ si $w \in L$, con la ayuda de un certificado x tal que $|x| \leq p(|w|)$. p es un polinomio, y de esta manera, se puede construir una MT M' que decida en tiempo $exp(|w|)$ si $w \in L$, sin usar ninguna cadena adicional: M' simplemente barre todos y cada uno de los certificados posibles x de w. Se pide explicar por qué M' efectivamente tarda tiempo $exp(|w|)$. Ayuda: como $|x| \leq p(|w|)$ y los símbolos de x pertenecen a un alfabeto de k símbolos, ¿cuántos certificados x puede tener a lo sumo una cadena w?

Dicho de forma más simple, el enunciado postula que si un lenguaje L está en NP, entonces existe una MT M que verifica si una cadena de entrada w pertenece a L en **tiempo polinomial** con la ayuda de un certificado x.

Pero si queremos decidir si w pertenece a L SIN usar un certificado en particular, entonces podemos construir una MT M' que construya a mano todos los posibles certificados x de w y verifique cada uno de ellos usando M, uno por uno.

Si el alfabeto Σ tiene k símbolos, entonces el número de posibles certificados x de longitud a lo sumo $p(|w|)$ es la suma de las potencias de k desde 0 hasta $p(|w|)$, es decir: $\sum_{i=0}^{p(|w|)} k^i = \frac{k^{p(|w|)+1}-1}{k-1} = O(k^{p(|w|)})$

Como se sabe que $p(|w|)$ es un polinomio, entonces $k^{p(|w|)}$ es una función **exponencial** con respecto a $|w|$. Por lo tanto, el número de certificados que M' tiene que verificar es de orden exponencial en el tamaño de la entrada w. Entonces M' tarda tiempo $exp(|w|)$.

5. Probar:

a. Si los lenguajes A y B son tales que $A \neq \emptyset$, $A \neq \Sigma^*$ y $B \in P$, entonces $(A \cap B) \leq_p A$.

1. Hipótesis:
   i. $A \neq \emptyset$
   ii. $A \neq \Sigma^*$
   iii. De lo anterior se deduce que existen como mínimo dos cadenas de entrada w_1 y w_2 tales que $w_1 \in A$ y $w_2 \notin A$.
   iv. $B \in P$, por lo cual existe una MT M_B que decide B en tiempo polinomial.
2. Se debe construir una MT M_f que compute una función de reducción polinomial f tal que:

   i. Si $w \in A \cap B$, entonces $f(w) \in A$.
   ii. Si $w \notin A \cap B$, entonces $f(w) \notin A$.
3. Dada una cadena de entrada w, primero se chequea si $w \in B$ usando M_B. Esto se hace en tiempo polinomial por hipótesis.
4. Si $w \notin B$, entonces se devuelve $f(w) = w_2$, con $w_2 \notin A$. Esto tarda tiempo polinomial porque es simplemente devolver una cadena de entrada fija.
5. Si $w \in B$, entonces se devuelve $f(w) = w$. Esto tarda tiempo polinomial porque es simplemente devolver la misma cadena de entrada.
   i. Si w pertenece a B pero no a la intersección $A \cap B$, entonces necesariamente w no pertenece a A.
   ii. Si w pertenece a B y además a la intersección $A \cap B$, entonces pertenece a A.
6. Por lo tanto se cumple que $w \in A \cap B$ si y solo si $f(w) \in A$, lo que implica que f es una función de reducción polinomial de $A \cap B$ a A. Por lo tanto, $(A \cap B) \leq_p A$.

b) Si $L_1 \in NPC$ y $L_2 \in NPC$, entonces $L_1 \leq_p L_2$ y $L_2 \leq_p L_1$.

1. Hipótesis:
   i. $L_1 \in NPC$, por lo cual $L_1 \in NP$ y para todo lenguaje $L' \in NP$, $L' \leq_p L_1$.
   ii. $L_2 \in NPC$, por lo cual $L_2 \in NP$ y para todo lenguaje $L'' \in NP$, $L'' \leq_p L_2$.
2. Como $L_1 \in NPC$ y $L_2 \in NP$, entonces por definición $L_2 \leq_p L_1$.
3. Como $L_2 \in NPC$ y $L_1 \in NP$, entonces por definición $L_1 \leq_p L_2$.
4. Por lo tanto, se cumple que $L_1 \leq_p L_2$ y $L_2 \leq_p L_1$.

c) Si $L_1 \leq_p L_2$, $L_2 \leq_p L_1$, y $L_1 \in NPC$, entonces $L_2 \in NPC$.

1. Hipótesis:
   i. $L_1 \leq_p L_2$
   ii. $L_2 \leq_p L_1$
   iii. $L_1 \in NPC$, por lo cual $L_1 \in NP$ y para todo lenguaje $L' \in NP$, $L' \leq_p L_1$.
2. Como $L_1 \leq_p L_2$ y $L_1 \in NP$, entonces por definición $L_2 \in NP$.
3. Por hipótesis 1 y 3, para todo lenguaje $L' \in NP$, $L' \leq_p L_1$ y como $L_1 \leq_p L_2$, entonces por transitividad de las reducciones polinomiales, $L' \leq_p L_2$.
4. Lo anterior cumple con la definición de NPC, por lo que $L_2 \in NPC$.

d) Si un lenguaje es NPC, entonces su complemento es CO-NPC, es decir, está en CO-NP y todos los lenguajes de CO-NP se reducen polinomialmente a él. Ayuda: $L_1 \leq_p L_2$ sii $L_1^C \leq_p L_2^C$.

1. Hipótesis: $L \in NPC$, por lo cual $L \in NP$ y para todo lenguaje $L' \in NP$, $L' \leq_p L$.
2. Como $L \in NP$, entonces $L^C \in \text{CO-NP}$.
3. Ahora se debe demostrar que para todo lenguaje $A \in \text{CO-NP}$, $A \leq_p L^C$.
   i. Sea $A \in \text{CO-NP}$. Por definición de CO-NP, $A^C \in NP$.
   ii. Como $L \in NPC$, todo lenguaje en NP se reduce polinomialmente a L, por lo que $A^C \leq_p L$.
   iii. Como se sabe que $L_1 \leq_p L_2$ sii $L_1^C \leq_p L_2^C$, entonces $A \leq_p L^C$.
   iv. Por lo tanto todo lenguaje en CO-NP se reduce polinomialmente a L^C.
4. Por lo tanto, L^C está en CO-NP y todos los lenguajes de CO-NP se reducen polinomialmente a él, lo que implica que $L^C \in \text{CO-NPC}$.

6. Sea el lenguaje $\text{SH-s-t} = \{(G, s, t) \mid \text{G es un grafo que tiene un camino de Hamilton del vértice s al vértice t}\}$. Un grafo $G = (V, E)$ tiene un camino de Hamilton del vértice s al vértice t sii G tiene un camino entre s y t que recorre todos los vértices restantes una sola vez. Probar

que $\text{SH-s-t} \in NPC$. Ayuda: se sabe que CH, el lenguaje correspondiente al problema del circuito hamiltoniano, es NPC.

Para probar que $\text{SH-s-t} \in NPC$, se deben probar dos cosas: 1) $\text{SH-s-t} \in NP$ y 2) $CH \leq_p \text{SH-s-t}$.

1. $\text{SH-s-t} \in NP$:
   ...
2. $CH \leq_p \text{SH-s-t}$:
   ...

7. Probar que el lenguaje $FACT = \{(N, M_1, M_2) \mid N \text{ tiene un divisor primo en el intervalo } [M_1, M_2]\}$ está tanto en NP como en CO-NP. Ayuda: Todo número natural N se descompone de una única manera en factores primos, los cuales concatenados no ocupan más de $poly(|N|)$ símbolos.

1. $FACT \in NP$:
   ...
2. $FACT \in \text{CO-NP} = FACT^C \in NP$:
   ...